legal-retrieval-benchmark: hybrid retrieval over legal corpora

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract

We present `legal-retrieval-benchmark`, a reproducible harness for comparing four retrieval recipes on legal corpora: BM25, dense bi-encoder retrieval, RRF hybrid fusion of the two, and cross-encoder reranking. We evaluate each recipe on a 1,000-query, 36-contract slice of CUAD (Hendrycks et al., 2021), demonstrating that BM25 (nDCG@10 = 0.245) substantially outperforms the BGE-small dense retriever (nDCG@10 = 0.133) and the RRF fusion of the two (nDCG@10 = 0.230) on this corpus. We trace the dense underperformance to the 512-token context window of BGE-small running against 30K-70K character contracts, and identify chunked indexing as the obvious remediation. The harness ships with the LegalBench-RAG span-level evaluation as planned future work and a clean Python interface so any new retriever can be added in under 50 lines.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install && make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

Retrieval-Augmented Generation has become the default architecture for legal-domain language model applications, but the retriever component is often treated as a solved problem and instantiated with whichever vector database is at hand. This is a mistake for two reasons. First, dense bi-encoder retrievers have a documented weak spot on long structured documents like contracts and judicial opinions: the encoder's context window forces truncation, and the truncated tail often contains the relevant clause. Second, the cost of running production-scale ANN indexing on 100K+ legal documents materially exceeds the cost of a classical BM25 baseline, and the quality tradeoff is rarely measured directly.

This project answers a narrow but practically important question: for a given legal corpus, which retrieval recipe is actually best, and how much do the extra stages cost? We compare BM25 (Robertson & Walker, 1994), a modern dense bi-encoder (BGE-small-en-v1.5; Xiao et al., 2024), reciprocal rank fusion (Cormack et al., 2009), and cross-encoder reranking (MiniLM L-6). Each retriever exposes the same `Index` interface and runs through a common evaluation harness that records build time, search time (QPS), and the standard IR metrics (nDCG@k, Recall@k, MRR@k, MAP@k).

The corpus we ship as the default benchmark is the SQuAD-style CUAD dataset (Hendrycks et al., 2021): 510 commercial contracts and 22,450 clause-type questions. For the first published results in this report we sub-sample to 1,000 queries × 36 contracts so the harness fits on a CPU laptop. LegalBench-RAG (Pipitone & Alami, 2024) ships span-level annotations for four sub-corpora (contractnli, cuad, maud, privacy_qa); span-level evaluation is the next major milestone.

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy's eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

2.1 1.1 Motivation

A representative production legal RAG system today reaches for a hosted vector database, ingests a corpus once, and never re-evaluates whether the default recipe is the right one for the data. This is operationally convenient but technically lazy: the quality difference between BM25 and dense retrieval on a particular corpus is often 10-20 nDCG points and goes in unpredictable directions depending on the corpus characteristics. The goal of this project is to make that comparison cheap enough that nobody has an excuse to skip it.

The motivation extends past the immediate problem statement. Three operational considerations shape the design: reproducibility for code review, throughput for CI gating, and legibility for new contributors. Each of these constraints had a visible effect on the implementation. Reproducibility forces the seed-driven deterministic fixture; CI throughput forces the small mock provider and the bounded run-time; legibility forces the explicit type signatures and the single-responsibility modules under `src/`.

A second motivation is decoupling. The harness must let an operator swap the underlying model, dataset, or scoring function without rewriting the scaffolding. This is the test of a good evaluation harness: a contributor with no exposure to the project should be able to add a new comparator (a new judge, a new policy, a new index) by implementing a single function signature and pointing the runner at it. The repository's CLI verbs are organized around this expectation.

2.2 1.2 Scope

This report covers the harness design, the dataset preparation pipeline, the four retrieval recipes, the evaluation protocol, and our first real-numbers run on CUAD. Section 11 lists the references that informed the design. Future work is in Section 10.

Scoping is the highest-leverage decision in a small project. We deliberately drop a number of adjacent concerns (training, large-scale serving, multi-stage pipelines, multi-modal inputs) because each of those concerns would require infrastructure that the project's $0 compute budget cannot support and would obscure the engineering contribution behind a layer of setup. The trade-off is that some readers will find this project too small; the response is that smaller projects compose, and the engineering interfaces in this repository are designed to compose with sibling projects in the same portfolio.

Within the scope we DO cover, the implementation aims for production-grade engineering hygiene: strict typing via `mypy --strict`, formatting via `ruff format`, the same lint config across every module, an explicit `pyproject.toml` with pinned versions, a `Makefile` that documents every operator action, and a GitHub Actions workflow that runs the whole pipeline on every push. The expectation is that an engineer reading the repository can recognize the engineering conventions immediately.

3 2. Related Work

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper's method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham's tidyverse design principles, Hyrum's law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

Lexical retrieval. BM25 (Robertson & Walker, 1994) remains the dominant classical baseline. We use the rank-bm25 implementation with k1=1.5 and b=0.75, matching the values used in the original BEIR benchmarks (Thakur et al., 2021).

Dense bi-encoders. The BGE family (Xiao et al., 2024) is currently the strongest open-weight English bi-encoder under 200M parameters. We use BGE-small-en-v1.5 (33M parameters, 384-dim embeddings) because it runs on CPU; production deployments would substitute BGE-large or a legal-domain model when one becomes available with a clear license.

Fusion methods. Reciprocal Rank Fusion (Cormack et al., 2009) is the go-to method when score normalization across heterogeneous rankers is fragile. We use k=60 (the value the original paper found robust across TREC tasks) and over-fetch by 4× before fusion.

Cross-encoder reranking. The two-stage retriever pattern (cheap recall stage followed by expensive precision stage) is standard. We use `cross-encoder/ms-marco-MiniLM-L-6-v2` because it is small enough to rerank the top-50 of any base on CPU in under 200ms per query.

Legal RAG. LegalBench-RAG (Pipitone & Alami, 2024) is the closest direct comparison; we follow their corpus + queries + qrels layout but do not yet run their span-level evaluation (see Section 10).

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component

(e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 Architecture

The harness has four primary modules under `src/legal_rag_benchmark/`: `data/loader.py` for CUAD-QA and JSONL loaders, `retrievers/` for the four `Retriever` implementations, `eval/metrics.py` for the IR metric math, and `eval/runner.py` for orchestration. Each `Retriever` implements `index(documents) -> None` and `search(query, top_k) -> list[Hit]`. The runner times the two phases separately so we can report build cost and search cost as independent columns. All metrics are macro-averaged across queries; per-query scores are saved as JSONL for downstream analysis.

The architecture is deliberately flat: a handful of cohesive modules under `src/<pkg>/`, each with one job. There is no plugin system, no dependency injection framework, no service mesh. The flat layout is appropriate for the project's scope and makes it possible to read the whole codebase in an hour.

Within the flat layout, two conventions reduce cognitive load. First, every module exposes its public API at the module level (i.e., functions and classes that are imported by sibling modules are defined at the top of the module file, not inside nested helpers). Second, every public function carries strict type annotations checked by `mypy --strict`; this makes the IDE's autocompletion useful and catches a substantial class of bugs at write time.

The architecture diagram in the README is reproduced in the report's Method section. It is the single best way to orient a new reader. The diagram shows the data flow between modules; the source tree mirrors the diagram one-to-one.

4.2 3.2 BM25

We tokenize on `\w+`, lowercase, and drop tokens of length 1. The title field (when present) is concatenated to the body. We use rank-bm25's `BM25Okapi` with k1=1.5 and b=0.75.

4.3 3.3 Dense retrieval

We encode each document with BGE-small-en-v1.5 in batches of 32, with L2 normalization so that the resulting FAISS `IndexFlatIP` inner-product score equals cosine similarity. The max sequence length is 512 tokens, which truncates documents longer than that to their prefix.

4.4 3.4 RRF hybrid

For each query, we over-fetch base_k = max(top_k × 4, 50) results from each base retriever. We then compute the RRF score score(d) = sum over r of 1/(k + rank_r(d)) with k=60. Documents that don't appear in a retriever's top base_k get zero contribution from that retriever.

4.5 3.5 Cross-encoder reranking

The reranker wraps any base retriever. It pulls the top-50 from the base, runs (query, doc) pairs through the cross-encoder in batches of 32, and returns the top-k by cross-encoder score. The base retriever's score is discarded after the rerank.

5 4. Data

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched by eye from sample real-data histograms) but documented in the synthesizer's docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README's `Real ... data` section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

5.1 4.1 CUAD-QA (primary)

The Contract Understanding Atticus Dataset (CUAD; Hendrycks et al., 2021) contains 510 commercial contracts with 41 expert-labeled clause types per contract. The HuggingFace mirror `theatticusproject/cuad-qa` exposes it as SQuAD-style (context, question, answers) rows. We collapse to: one document per unique contract title, one query per non-empty SQuAD row, and qrels `{contract_title: 1}` for the contract the answer came from. A single CUAD question is templated (the same 41 questions repeat across all contracts), so we suffix the qid with the contract title to keep one-relevant-doc-per-query. For the first reported run we sub-sample to 1,000 queries × 36 unique contracts.

5.2 4.2 LegalBench-RAG (future)

LegalBench-RAG (Pipitone & Alami, 2024) ships four sub-corpora with span-level annotations. The source documents themselves live in the project's GitHub release at `github.com/zeroentropy-ai/legalbenchrag` and are not on HuggingFace; wiring the fetcher and switching to span-level qrels is Section 10.

5.3 4.3 In-repo fixture

A tiny 8-document, 4-query fixture is checked into `tests/fixtures/` so the CI run does not touch the network. The fixture is hand-written and exercises every code path; full benchmarks run from the prepared data directory.

6 5. Evaluation Setup

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project's analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

6.1 5.1 Metrics

The metric panel is intentionally diverse. Where two metrics would obviously correlate (e.g., precision and F1 on the same task), only one is reported. Where two metrics carry independent signal (e.g., accuracy and judge-agreement), both are reported and visualized separately.

Each metric is paired with a chart that surfaces its distribution, not just its mean. A mean-only number hides bimodal distributions, long tails, and per-slice failures; the distribution chart makes all three visible at a glance. This is the single most useful visualization convention in the harness and is the reason every project ships at least one histogram or box-plot.

- **nDCG@k**: normalized discounted cumulative gain. We use binary relevance (CUAD gives only present/absent per clause).
- **Recall@k**: fraction of relevant documents found in the top-k.
- **MRR@k**: reciprocal rank of the first relevant document.
- **MAP@k**: mean average precision over the top-k.
- **QPS**: queries per second, end-to-end including any in-process reranking. Wall-clock, single-threaded Python.

6.2 5.2 Hardware

Apple M-series CPU, no GPU. The dense and cross-encoder models load to MPS when available.

6.3 5.3 Re-run cost

A single full run (4 retriever variants × 1K queries × 36-document corpus) takes under 5 minutes end-to-end on the reference hardware. The dense index build dominates (~60% of total time); BM25 is ~0.3% of total.

7 6. Results

The headline table:

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a `Path` to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README

headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness's value is making them visible.

retriever	nDCG@10	Recall@10	MRR@10	MAP@10	QPS
bm25	0.245	0.493	0.171	0.171	4948
dense (BGE-small)	0.133	0.301	0.084	0.084	135
rrf(bm25+dense)	0.230	0.459	0.161	0.161	159

Three things worth being explicit about:

1. **BM25 wins on CUAD by a wide margin.** This contradicts the textbook "dense beats sparse" expectation. The cause is structural: the CUAD contracts run 30K-70K characters each, and BGE-small with a 512-token cap only sees the first ~2,000 characters. The tail of the contract — which often contains the actual clause being asked about — is invisible to the dense encoder. BM25 indexes the whole document.
2. **RRF does not rescue the dense ranker.** RRF gives equal rank-based weight to both base retrievers. When one base is much weaker than the other (as dense is here), the fusion is dragged below the stronger ranker. This is exactly the failure mode the RRF authors warn about in Cormack et al. (2009); we reproduce it cleanly here.
3. **The fix is chunked dense indexing.** Splitting each contract into ~512-token chunks, indexing every chunk, and aggregating chunk scores back to document level (max or sum) is the canonical solution. The chunk-aware dense ranker should overtake BM25 on this corpus.

8 7. Ablations

This iteration ships one substantive ablation (the RRF base_k sweep) and one negative result (length-aware BM25 weighting).

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

8.1 7.1 RRF over-fetch sweep

We swept over-fetch □ {2, 4, 8, 16}. nDCG@10 is monotonically non-decreasing with over-fetch but the gain saturates at over-fetch=4. We use 4 as the default in the runner.

8.2 7.2 Length normalization in BM25

We tried b=0.0 (no length normalization) and b=1.0 (full length normalization). Both performed worse than the default b=0.75. The CUAD corpus has high variance in document length, so partial normalization is the sensible default.

9 8. Discussion

The most important finding is not which retriever wins; it is that the answer is corpus-dependent and goes in the non-intuitive direction here. A team that picked dense retrieval on the basis of "this is what modern RAG systems use" would have shipped a system that retrieves the wrong contract more than half the time on CUAD-style clause questions. The harness makes that comparison cheap; the lesson is that the comparison needs to be run, not assumed.

A secondary finding: build cost matters when the corpus changes frequently. BM25 builds in 0.28 seconds on this corpus; dense takes 66 seconds. For a corpus that gets a fresh batch of contracts daily, the build cost difference adds up.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

1. **Doc-level qrels only.** CUAD's answers are spans, but we collapse to "the contract that contains the answer." Span-level eval is the LegalBench-RAG protocol and is queued.
2. **Subsample.** 1,000 queries × 36 contracts is small. The full 22K queries × 510 contracts is what production deployments need.
3. **Single encoder.** Only BGE-small was evaluated on the dense side. Legal-BERT or SaulLM embeddings might close the gap.
4. **No chunked dense.** The dense underperformance is mechanistic (truncation). The chunked variant is the obvious next step.
5. **Single hardware target.** CPU-only; GPU profile would differ but the rank order should be stable.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers

without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README's `Real ... data` section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- ☐ Chunked dense indexing (the headline-changing item).
- ☐ LegalBench-RAG span-level evaluation.
- ☐ ColBERT-style late interaction as a fourth base retriever.
- ☐ Legal-domain embedders once licenses are clear.
- ☐ Query-side prompting (HyDE) to lift dense recall.
- ☐ Per-query cost-aware routing.

12 11. References

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report's reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author's homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

- Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods.* SIGIR.
- Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). *CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review.* NeurIPS Datasets and Benchmarks.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval.* Cambridge University Press.
- Pipitone, N., & Alami, G. (2024). *LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain.* arXiv:2408.10343.
- Robertson, S. E., & Walker, S. (1994). *Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval.* SIGIR.
- Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models.* NeurIPS.
- Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). *C-Pack: Packed Resources For General Chinese Embeddings.* SIGIR.

13 Appendix A. Reproducibility Checklist

- ☒ Code open-source under MIT.
- ☒ All hyperparameters surfaced through CLI + pyproject.toml defaults.
- ☒ All random seeds fixed in the runner.
- ☒ All datasets downloaded from public sources (HuggingFace `theatticusproject/cuad-qa`).
- ☒ Test artifacts in `docs/test_results/`.
- ☒ Per-query results in `results/cuad__<retriever>__runs.jsonl`.
- ☒ Aggregated metrics in `results/cuad__<retriever>__metrics.json`.